

● PoC 実装済み — Phase 7 完了

Playwright ベースの 自作 Figma MCP

ガバナンス制約を回避する、
安全なデザイン参照の新アプローチ

デジタルプロダクトデザインチーム ・ 2026

```
figma-mcp-service

// GitHub Codespaces VM
const img = await mcp.call({
  tool: "figma_get_screenshot",
  url: "figma.com/design/...",
});

// ✓ Figma export API: 不使用
// ✓ 公開 S3: 経路ゼロ
// ✓ bytes: in-process のみ
// → base64 PNG を AI に直接渡す
```

背景と課題 — なぜ公式 MCP が使えないのか

公式 MCP のデータフロー

- 01 AI が `figma_get_image` を呼ぶ
- 02 Figma の画像エクスポート API が発火
- 03 PNG が公開 S3 に書き込まれる
- 04 30日間、URL を知れば誰でも取得可能

何が問題なのか



公開 S3 に 30日保持

`figma-alpha-api.s3.us-west-2.amazonaws.com`
URL を知れば誰でもアクセス可能



TTL・アクセス制御が不能

Figma 側が握る S3 のため、こちら側で
ポリシーを変更できない



社内ガバナンスに抵触

デザイン資産が外部の公開ストレージに残る
→ 社内で全面禁止

仮説 — エクスポート API を使わなければならない

公式 MCP の問題は「Figma のエクスポート API を叩くこと」そのものにある。
エクスポート API を一切使わず、Figma の Web キャンバスをブラウザで描画して
その画素を直接撮れば、公開 S3 経路は構造的に存在しなくなる。

Playwright (ヘッドレス Chrome) を VM で動かし、Figma Web をそのままレンダリング → スクショ → AI へ直接渡す



公開 S3 経路がゼロ

エクスポート API を一度も叩かない
→ S3 にファイルが生成されない



バイトはメモリ内で完結

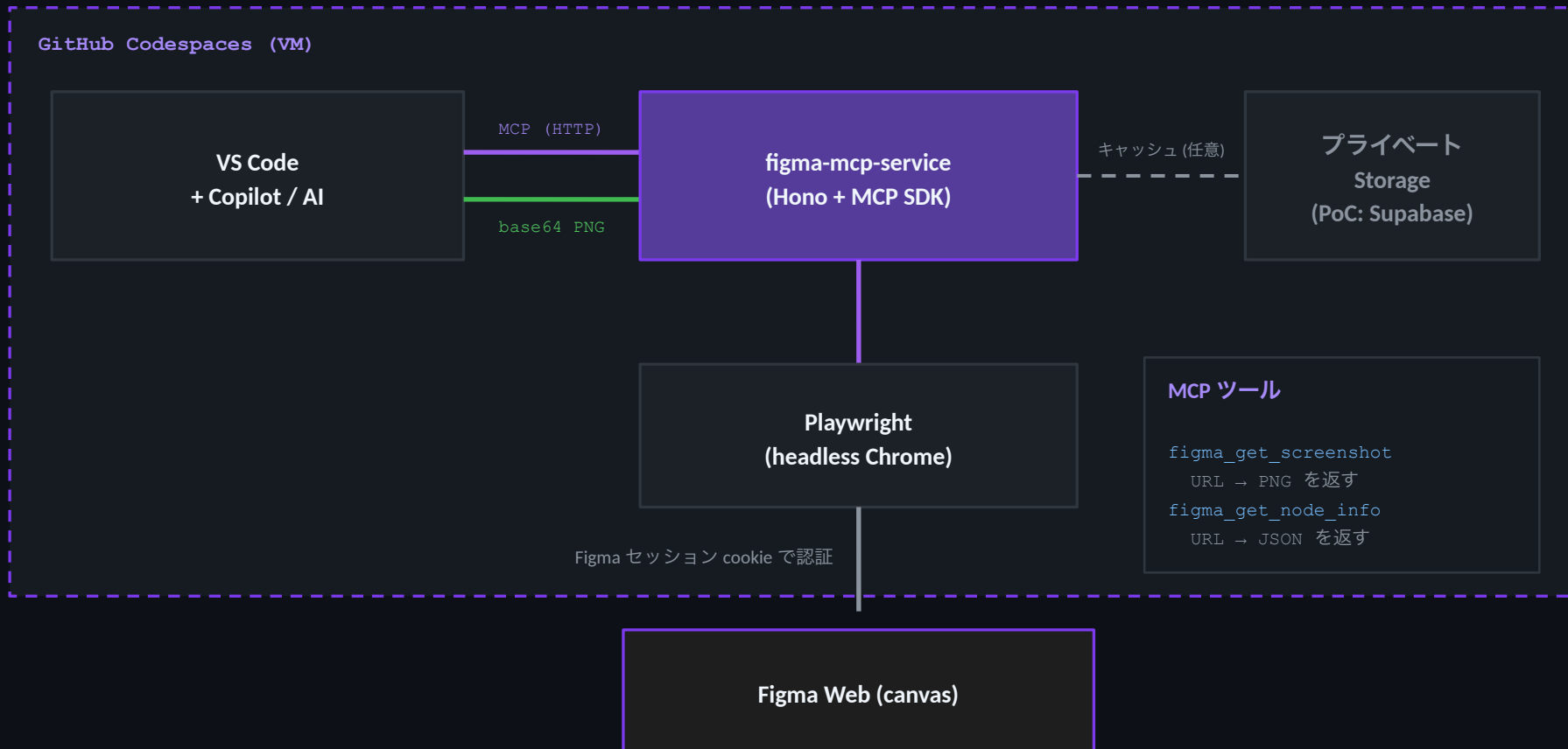
画像は VM プロセス内のみ
中間ストレージを経由しない



Figma 純正の忠実度

Figma Web アプリが直接描画
→ 見た目の忠実度は 100%

提案する構成 — アーキテクチャ概要



セキュリティの肝 — なぜこれがセキュアなのか

	公式 MCP		この構成
画像の出力先	公開 S3 (30日保持)	→	VM プロセス内のメモリのみ
画像の流れ	エクスポート API → 公開 URL	→	Playwright → MCP → AI に直接
公開 URL	生成される	→	一切生成しない
キャッシュ	制御不能	→	会社管理のプライベートストレージ
アクセス制御	なし	→	ユーザ単位の分離 (RLS)

不変条件: スクショのバイト列は Playwright が撮った瞬間からメモリ上にあり、AI に渡るまで公開 URL を一度も経由しない

PoC は既に動いている — 実装状況

✓ Phase 7 完了

技術スタック

Hono v4.6

HTTP サーバ / ルーティング

MCP SDK v1.0

MCP サーバ (HTTP transport)

Playwright v1.49

ヘッドレス Chrome 自動化

Supabase

Auth / Postgres / Storage (PoC 用)

Zod

環境変数・リクエストの検証

Playwright レンダリングフロー

- 1 storageState から Figma セッションを復元
- 2 ヘッドレス Chrome 起動 (ボット検知回避)
- 3 figma.com/design/{fileKey} に遷移
- 4 Shift+0 でノードにズーム
- 5 canvas の bounding box を取得
- 6 PNG でスクショ → バイトをメモリに保持
- 7 base64 化して MCP レスポンスに返却

PoC 検証結果 — ログに公開 URL は出てこない

実際に PoC を動かしたログ。公式 /v1/images は公開 S3 URL を露出するが、この構成のログには URL が一切出ない。

● 公式 MCP ・ demo-tier-c-leak.sh

```
$ ./demo-tier-c-leak.sh

# Step 1 — /v1/images で URL 発行
"images": {
  "1:23": "https://figma-alpha-api
    .s3.amazonaws.com/img/8f2c..."
}
```

```
# Step 2 — 認証ヘッダなしで GET
HTTP status: 200 ← 誰でも取得可
PNG size: 184320 bytes
```

× 公開 S3 URL がログに残る

● この構成 ・ figma-mcp-service

```
$ figma_get_screenshot <figma URL>

{"msg":"render.playwright_done",
  "bytes":248914}
{"msg":"render.bytes_loaded",
  "source":"playwright.inline",
  "signedUrlCreated":false
{"msg":"render.complete",
  "cacheHit":false,"durationMs":8423}
```

```
$ grep -c "amazonaws\\|https" *.log
→ 0 ← URL は一行も出力されない
```

✓ ログにも公開 URL は存在しない

結論 公式 MCP はログにすら公開 S3 URL が残るが、この構成は render ログを grep しても URL 0 件 — バイトはプロセス内で完結している。

Supabase の役割と本番への移行方針

Supabase は PoC 専用の足場 — Auth + Postgres + Storage の詰め合わせ。本番では会社の基盤に置き換える。



現在 Fly.io + Supabase で検証中。社内環境へのアクセス権取得後、会社の基盤に移植予定。

デプロイ構成 — PoC → 本番 移行マップ

	PoC (検証用) — Fly.io / Supabase	本命 — 社内クラウド基盤
MCP + Playwright	Fly.io (常駐コンテナ)	→ ECS Fargate / Azure Container Apps
認証	Supabase Auth (JWT)	→ 会社 IdP (OIDC) — 最優先
画像キャッシュ	Supabase Storage	→ プライベート S3 / Azure Blob
メタデータ DB	Supabase Postgres	→ RDS / Azure Database
監査ログ	audit_log テーブル	→ 会社のログ基盤 (SIEM 等)
キャッシュ削除	SQL 関数 cron	→ Lambda / Azure Functions Timer

最小構成: コンテナ 1台 + 会社 IdP + ローカルディスクキャッシュ + 標準出力ログ — Storage も DB も不要。スケール要件が出た時点で追加。

未検証のポイントとリスク

MED

Codespaces での Playwright

ヘッドレス Chrome が Codespaces VM で安定稼働するか要検証 (メモリ / --no-sandbox 前提)

HIGH

AI へのデータ送信

公開 S3 漏洩は防げるが、スクショをモデルプロバイダに送信する事実はガバナンス整理が必要

MED

Figma セッション管理

storageState (cookie) の保持場所と更新。
Codespaces は ephemeral。env 渡し対処済み

LOW

ボット検知

Figma 側の検知強化で UA 偽装等が効かなくなる可能性 (現状は回避実装済み)

LOW

Figma 利用規約

Playwright での自動操作が Figma ToS と整合するか、法務・セキュリティに確認要

LOW

スケール

MCP セッションはプロセス内メモリ。水平スケールには Redis 等が必要 (現状は単一プロセス)